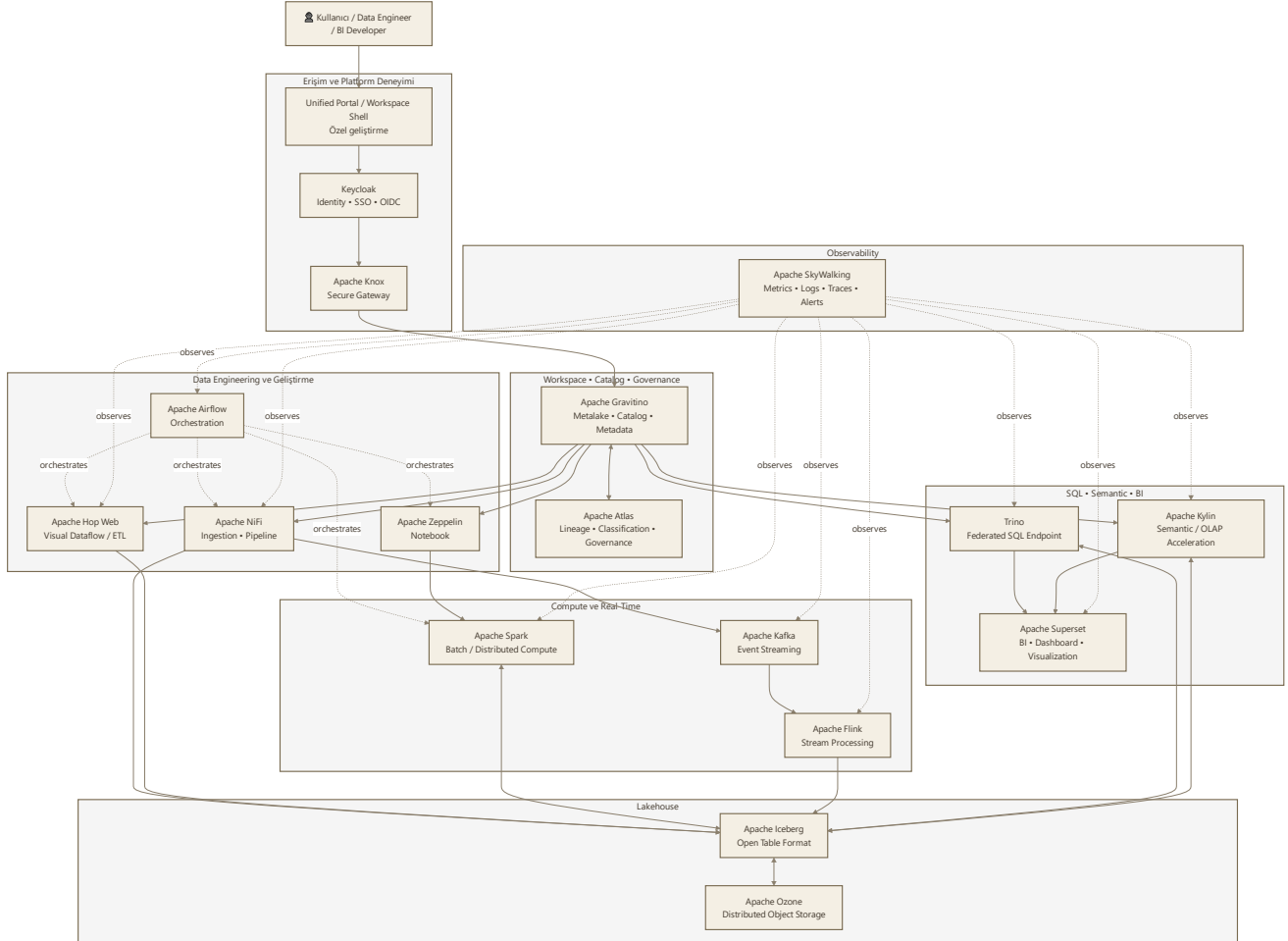


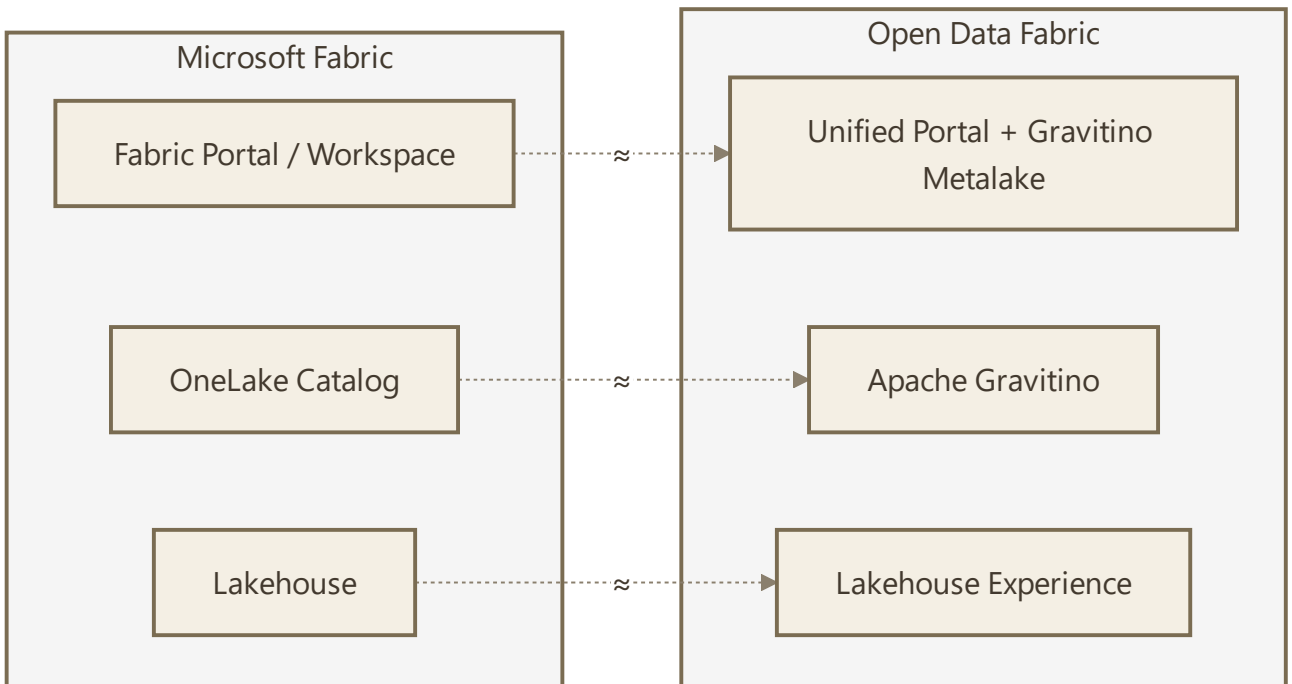
Open Data Fabric — Apache Ağırlıklı Referans Mimari

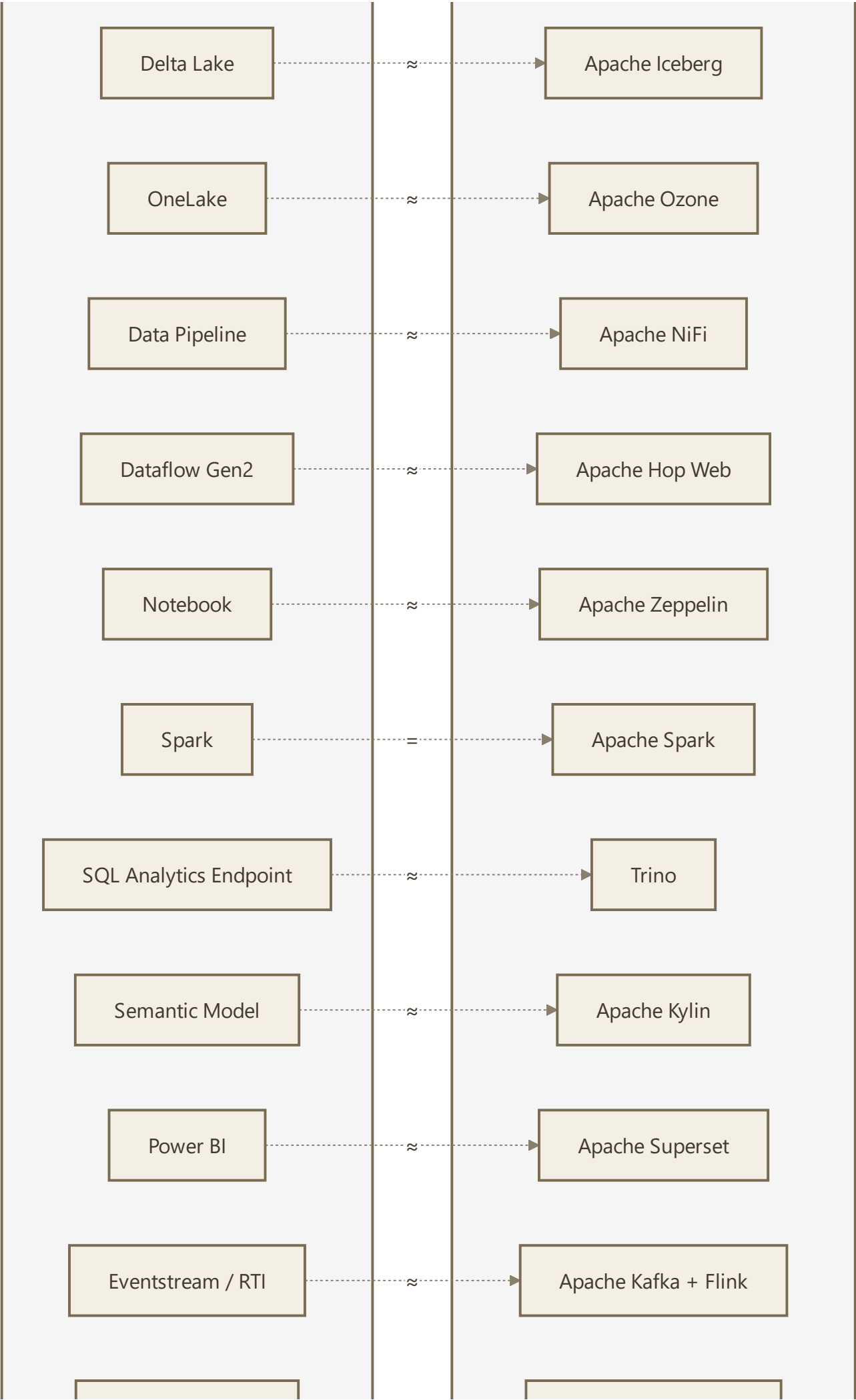
Amaç: Microsoft Fabric'teki temel deneyimleri mümkün olduğunca Apache/açık kaynak bileşenlerle karşılayan, on-prem / cloud / hybrid çalışabilen birleşik veri platformu.

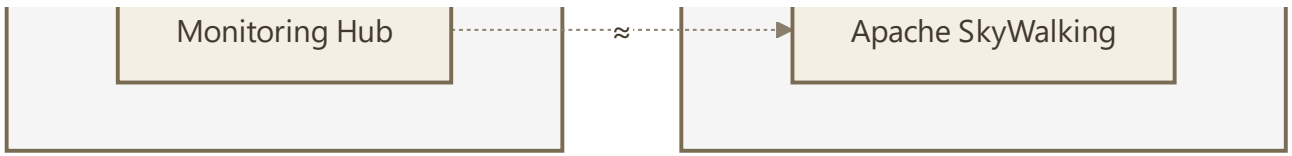
1. Üst Seviye Mimari



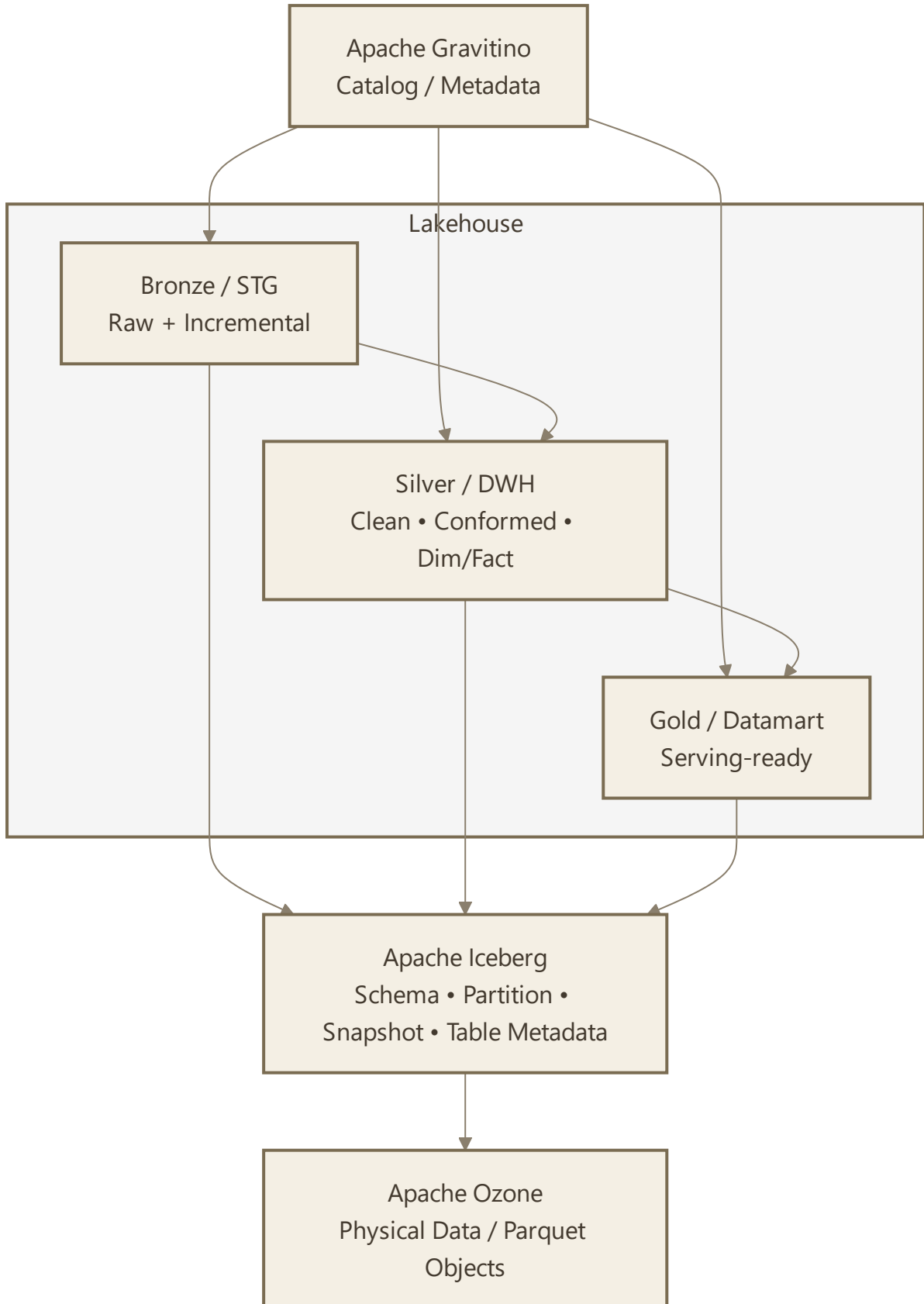
2. Fabric ile Katman Eşleştirmesi



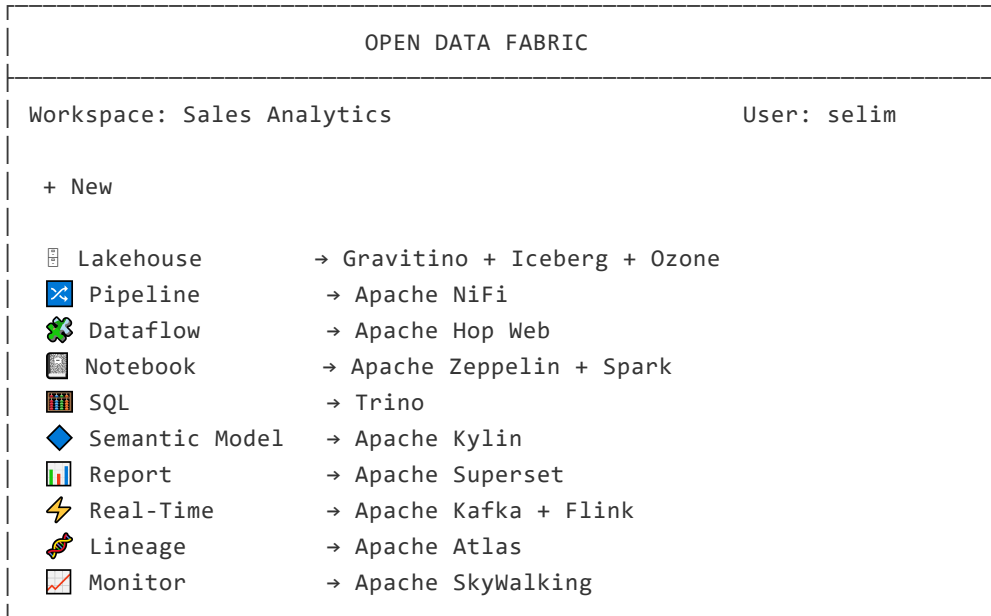




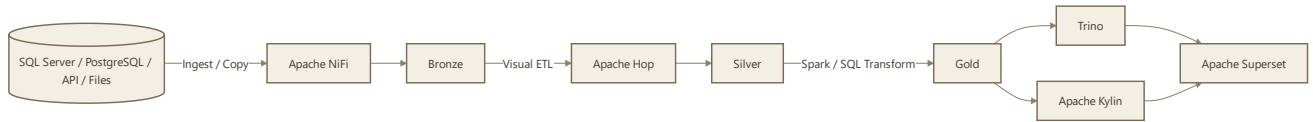
3. Lakehouse İç Yapısı



4. Kullanıcı Deneyimi — Fabric Benzeri Portal



5. Veri Akışı Örneği



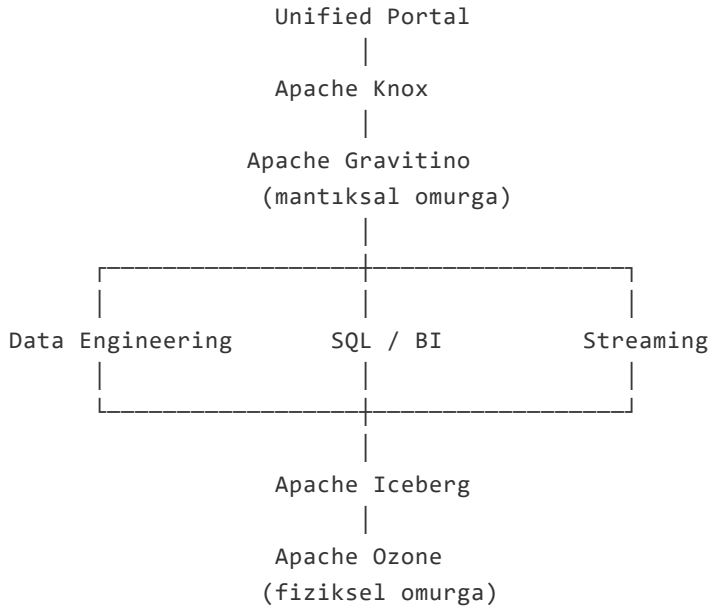
6. Bileşenlerin Sorumlulukları

Katman	Ürün	Ana sorumluluk
Portal	Özel geliştirme	Tek giriş noktası, workspace ve Fabric-benzeri UX
Identity	Keycloak	Kullanıcı, grup, SSO, OIDC/OAuth2
Gateway	Apache Knox	Veri platformu servislerine güvenli erişim
Catalog	Apache Gravitino	Metalake, catalog, schema, table ve metadata omurgası
Governance	Apache Atlas	Lineage, classification ve governance
Storage	Apache Ozone	Dağıtık fiziksel object storage
Table Format	Apache Iceberg	ACID lakehouse tabloları ve metadata
Ingestion	Apache NiFi	Kaynaklardan veri taşıma ve akış tabanlı pipeline
Visual ETL	Apache Hop Web	Dataflow Gen2 benzeri görsel dönüşüm
Orchestration	Apache Airflow	DAG, dependency, schedule ve job orchestration
Notebook	Apache Zeppelin	Web notebook ve interaktif geliştirme
Compute	Apache Spark	Büyük ölçekli batch / distributed processing
SQL	Trino	Federated/ad-hoc SQL endpoint
Semantic / OLAP	Apache Kylin	Semantic/OLAP model ve query acceleration

BI	Apache Superset	Dashboard, rapor ve visualization
Event Streaming	Apache Kafka	Event ingestion / event bus
Stream Compute	Apache Flink	Stateful real-time processing
Observability	Apache SkyWalking	Metrics, logs, traces, alerts ve topology

7. Tasarım İlkesi

Platformun merkezi yalnızca storage değildir:



Gravitino = mantıksal metadata/catalog omurgası

Iceberg = lakehouse tablo katmanı

Ozone = fiziksel veri omurgası

Portal = bütün bileşenleri tek ürün deneyiminde birleştiren katman

8. Fabric'e Göre En Önemli Fark

Microsoft Fabric'te portal, identity, catalog, storage, compute, semantic model ve BI tek yönetilen ürün deneyimi içinde gelir. Bu mimaride teknik motorların büyük bölümü hazır açık kaynak projelerden oluşur; bizim geliştirmemiz gereken temel değer **bu motorları tek workspace, ortak yetkilendirme, ortak metadata ve tutarlı kullanıcı deneyimi altında birleştirmektir**.

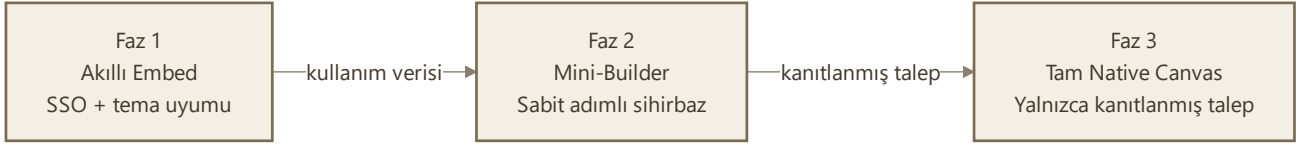
9. Portal Entegrasyon Modeli — Sürükle-Bırak Deneyimi

Fabric'te kullanıcı Dataflow Gen2'yi tek bir portal içinde, ayrılmadan sürükleyip bırakır. Bu mimaride sürükle-bırak canvas'ları (Apache Hop Web, Apache NiFi) zaten mevcut ama her biri kendi ayrı arayüzünde çalışıyor. "Unified Portal / Workspace Shell" henüz **özel geliştirme** olarak işaretli olduğundan, bu iki canvas'ın Portal'a nasıl bağlanacağı netleştirilmelidir.

Neden sıfırdan native editor önerilmiyor

Bir node-based flow editor'ü (React Flow / Rete.js vb. ile) sıfırdan geliştirmek; connector şeması, validation, execution engine entegrasyonu, versiyonlama gibi Hop ve NiFi'nin yıllardır olgunlaştırdığı yetenekleri yeniden inşa etmek anlamına gelir. Bu, dokümanın 8. bölümündeki "asıl değerimiz motorları birleştirmektir" ilkesiyle çelişir.

Önerilen üç fazlı yol



Faz	Kapsam	Efor	Ne zaman
1 — Akıllı embed	Hop Web ve NiFi, Portal içine iframe olarak gömülür; Keycloak OIDC token'ı ile SSO session otomatik geçirilir; Portal navigasyonu/teması CSS override ile uyumlulaştırılır	Düşük	Baştan itibaren
2 — Mini-builder	Genel amaçlı editör değil, kısıtlı ve hedefe özel bir sihirbaz (ör. "kaynak seç → dönüştür → hedef seç" 3 adım); arkada NiFi/Hop API'sine flow tanımı gönderilir	Orta	Kullanıcı araştırması "basit senaryolar Portal'dan çıkmadan olsun" derse
3 — Tam native canvas	Portal'ın kendi sürükle-bırak editörü, Hop/NiFi API'lerini arkada çağırır	Yüksek	Yalnızca Faz 2 kullanım oranı yüksekse ve gelişmiş senaryo talebi kanıtlanmışsa

Sonuç: Fabric'in "tek arayüz" hissi büyük ölçüde entegrasyon kalitesinden (SSO, tutarlı tema, hızlı geçiş) gelir, illa tek kod tabanından değil. Faz 1 iyi yapıldığında kullanıcı deneyimi farkı çoğu senaryoda fark edilmeyecek kadar küçülür ve aylarca sürececek bir editor geliştirme riski baştan alınmamış olur.

10. Revizyon Notları — Önerilen Alternatifler

İhtiyaç	Mevcut öneri	Neden yetersiz kalabilir	Önerilen	Ne sağlıyor
Gateway	Apache Knox	Kerberos SPNEGO odaklı; OIDC-only ortamda gereksiz operasyonel yük, her servis için özel topology tanımı gerekir	APISIX / Envoy + OIDC plugin	Keycloak ile doğrudan, deklaratif route tanımı, rate-limiting ve metrics kutudan çıkar
Lineage	Apache Atlas (tek başına)	Hook'lar Hive/HBase/Kafka kökenli; Spark/Trino/Iceberg için resmi/bakımlı hook yok	OpenLineage → Atlas'a besleme	Spark/Trino/Airflow'da olgun native destek; run-level lineage otomatik toplanır
Semantic model	Apache Kylin	Kylin bir OLAP pre-aggregation motoru, Power BI Semantic Model'in ilişki/ölçü/DAX katmanına karşılık gelmiyor; Iceberg desteği sürüme bağlı	dbt Semantic Layer / Cube (Kylin'in yanında, yerine değil)	Versiyonlanmış, tek doğruluk kaynağı metrik tanımları; Kylin hesaplama motoru olarak kalabilir

Observability	SkyWalking (her bileşen için)	Mikroservis/tracing odaklı; Kafka/Spark/Trino/Kylin için olgun plugin seti yok	Prometheus + Grafana + Loki/Tempo (SkyWalking yalnızca Portal için)	Hazır JMX exporter'lar ile sıfır ek geliştirmeyeyle dashboard
Metadata omurgası	Apache Gravitino	NiFi/Hop/Zeppelin ile native entegrasyon yok (yalnızca Trino/Spark/Flink olgun)	Değişiklik yok — ayrı iş kalemi olarak işaretlenmeli	Gerçekçi efor planlaması

Not: Trino ve Keycloak Apache Software Foundation projesi değildir; mevcut tasarımda teknik ihtiyaç nedeniyle bilinçli olarak korunmuştur.